The Benchmark Chooses the Winner

An Annotated, Plain-Language Edition

Measuring what fine-tuning does to small safety guards — and a cheap fix

Reza Rahimi
JazzX AI, Los Altos, CA
reza.rahimi@jazzx.ai

Abstract

Teams often take a small chat model, do a quick fine-tune to turn it into a *safety guard* (a filter that flags harmful prompts), and report how well it scores on a public benchmark. This edition re-tells a study that asks a sharper question: *what did the fine-tune change compared to the exact same model before fine-tuning, and does that change hold up on datasets the model never trained on?* On four small instruction models (1.5–4B parameters), each fine-tuned five times with LoRA, we find a clear split. On test data from the **same datasets used in training** ("represented" sources), fine-tuning is a large, reliable win: the main score rises by about +0.33 (to ≈ 0.98 for every model). On **four brand-new datasets** ("transfer"), the same fine-tuning is a small average loss (about -0.05), and at a deployed threshold it catches about *one in five fewer* attacks on a hard held-out harm set (HarmBench recall $78.4\% \to 57.5\%$). Crucially, the average hides a pattern: fine-tuning *helps weak base models and hurts strong ones* on new data. The lesson — and the title — is that **whichever benchmark you choose to report decides whether fine-tuning "looks like" a win**. We then show an early, cheap *remedy*: instead of tuning harder, keep the untuned base in the decision — averaging the base and the tuned guard's scores (a two-model "ensemble") keeps almost all the in-domain gain *and* recovers much of the lost generalization, most where fine-tuning hurt most. Every concept needed to read this (guard scoring, LoRA, average precision, the bootstrap, calibration, ensembling) is taught inline. The numbers are a retrospective "legacy" run and are reported as estimates with uncertainty ranges, not as proven claims; a clean re-run would be needed to confirm them. This is a plain-language companion to the formal paper, which remains the source of truth wherever this edition simplifies.

Contents

1 Introduction

A **prompt-safety guard** reads a user's prompt and decides whether it is okay (**safe**) or an attack or harmful request (**unsafe**). "Unsafe" here means one of three things: *harmful content* (asking for something dangerous), a *jailbreak* (trying to trick the model into ignoring its rules), or a *prompt injection* (hiding instructions in the input to hijack the system). Guards are attractive because they are small and cheap to run in front of a bigger model.

The common recipe is: take a general chat model, fine-tune it on your own labeled prompts, and report its score on a benchmark. The trouble is that a benchmark score, on its own, hides the quantity a practitioner actually needs: *what changed relative to the same model before fine-tuning, and does that change generalize?* If you only compare your tuned guard against *other* models, a gain on the datasets you trained on can masquerade as a broadly smarter guard.

Background you'll need — what a guard is and how we score it

The guard is just a language model asked to answer with one word: `safe` or `unsafe`. Instead of letting it write a sentence, we do **one forward pass** and look only at the two candidate next-words. For each, the model produces a raw preference number (a **logit**). The score is the gap between them,

$$s(x) = z_{\text{unsafe}} - z_{\text{safe}},$$

positive meaning "leans unsafe." A **softmax** over just those two turns the pair into a probability that adds up to 1. *Mini-example:* if $z_{\text{unsafe}} = 2.0$ and $z_{\text{safe}} = 1.0$, then $s = 1.0$ and $P(\text{unsafe}) = e^{2.0}/(e^{2.0} + e^{1.0}) \approx 0.73$. Ranking prompts by this score is all the guard has to do.

The question. For a fixed panel of four checkpoints and one frozen LoRA recipe, does fine-tuning change a guard's discrimination (how well it separates unsafe prompts from safe ones) *differently* on datasets represented during training versus datasets held out from training?

Contributions. (1) A controlled, same-model comparison: every fine-tuned guard is measured against *its own* untuned base on identical prompts, metrics, and scoring. (2) A separation of two effects that are usually blended into one leaderboard number: gains on *represented* sources versus *transfer* to held-out datasets, with per-model and per-benchmark detail. (3) A fully auditable release: hashes, seed-level results, the scoring table, and generated tables/figures.

Scope. English input-prompt classification only; a binary safe/unsafe decision; small models (1.5–4B); one LoRA recipe; three represented datasets; four transfer datasets; two single-class stress sets; four fixed checkpoints. It is *not* a leaderboard, a fine-tuning-objective comparison, an ensembling or fairness study, or a domain-compliance case study.

2 Background you'll need

This section teaches the five ideas the results rest on. If you already know average precision, the bootstrap, and calibration, skip to Section 3.

2.1 LoRA supervised fine-tuning (SFT)

Background you'll need — LoRA-SFT

LoRA fine-tuning [10] does **not** create a new model. The original model is **frozen** (no weight changes). Instead you bolt on a small set of extra trainable weights — an **adapter** — plugged into specific matrices inside the model. Only the adapter learns, so it is cheap: you train a small fraction of extra parameters (millions, not billions) and ship a small file that snaps onto the frozen base. "Supervised" means training on labeled examples (prompt + correct verdict word). **Completion-only loss on the verdict token** means the model is graded *only* on producing the single answer token, not on echoing the prompt — so all the learning pressure lands on that one decision.

2.2 Average precision (AP), and why not accuracy

Background you'll need — average precision

The guard outputs a *score* (the logit difference), and a good guard gives *higher* scores to truly-unsafe prompts. Two everyday quantities describe how it does: **precision** = of the prompts flagged so far, the fraction that are truly unsafe; and **recall** = of all the unsafe prompts, the fraction we have flagged. **Average Precision (AP)** rolls these up with **no threshold to pick**: it is the area under the precision–recall curve (0–1, higher better) — in words, how well the guard *ranks* unsafe prompts above safe ones. Why not plain **accuracy**? Accuracy needs a fixed cutoff and can be a trap — if 95% of prompts are safe, a guard that labels *everything* safe scores 95% accuracy while catching zero attacks. AP does not fall for that. *Mini-example:* two unsafe and three safe prompts, ranked {unsafe, safe, unsafe, safe, safe}. Precision at the first unsafe $= 1/1 = 1.0$ (1 unsafe among the top 1); at the second $= 2/3 \approx 0.67$ (2 unsafe among the top 3); AP $= (1.0 + 2/3)/2 \approx \mathbf{0.83}$ (a perfect ranking scores 1.0). *Tie-aware* means two prompts with the same score get fair, not lucky, credit.

Throughout, the headline metric is **benchmark-macro AP**: average the AP across the datasets in a group so each dataset counts equally ("macro"), then average across the four models, and for fine-tuned guards across the five seeds too.

2.3 Represented sources vs. held-out transfer

Background you'll need — the seen-vs-unseen distinction (the crux)

Represented sources are fresh, held-out rows from the *same* datasets used in training (same style and labeling habits, different examples). Like taking this year's exam on a topic you crammed from last year's — you do well, partly because you learned the test's patterns. **Transfer** is a *new-material exam*: four entirely different datasets never used in training. Reporting a single mixed number hides the difference between "got better at *these* datasets" and "got broadly better" — exactly the confusion this paper is about. *(Honest caveat: the transfer sets were looked at during development, so they are "dataset-held-out," not a sealed surprise.)*

Table 1: The fixed four-checkpoint panel and the one LoRA-SFT recipe used for every cell.

Checkpoint	Params	Role
Qwen2.5-1.5B-Instruct	1.5B	base → 5 SFT seeds
SmolLM2-1.7B-Instruct	1.7B	base → 5 SFT seeds
SmolLM3-3B [1]	3B	base → 5 SFT seeds
Qwen3-4B	4B	base → 5 SFT seeds
Recipe: LoRA r=32, α=64, dropout 0.05 on q,k,v,o,gate,up,down; completion-only loss; 1,200 training rows; 300 steps; learning rate 2×10^{-4} cosine; effective batch 4; max length 1024.		

2.4 The bootstrap and confidence intervals; "descriptive, not proven"

Background you'll need — bootstrap confidence intervals

Each headline number is one estimate from the few models and seeds we ran; a different draw could land higher or lower. To see how much it would *wobble*, we **resample our own results** (with replacement) 10,000 times and recompute the average each time (a **paired hierarchical bootstrap**: *paired* = base vs. fine-tuned, to measure the *change*; *hierarchical* = resample at two levels, which seeds count and — because the eval sets contain many near-duplicate prompts — how much each group of duplicates counts; the four models are held fixed). The middle 95% of those recomputed averages is the **two-sided 95% confidence interval (CI)** — a plausible range for the true change. A one-sided **LCB** says "at least this much"; a **UCB** says "no more than this." **Why no "statistically significant"?** These scores are a *legacy* artifact computed before the analysis was locked; declaring significance on already-peeked data is misleading. So the study stays **descriptive**: it reports ranges, makes no pass/fail verdict, and treats directions as *suggestive* pending a clean, pre-registered re-run.

2.5 Calibration and the operating point

Background you'll need — calibration, thresholds, TPR and FPR

AP is threshold-free, but to *deploy* a guard you must draw a line. First, **calibration (temperature scaling)**: divide the score by one tuned number so the stated probabilities match reality (this does not reorder prompts). Second, a **threshold**: here a *conservative cap* chosen so at most ∼5% of genuinely safe prompts are flagged (the target false-positive rate); because it is conservative, the realized rate can land well below 5%. Then you *measure* on test data: **TPR** (recall) = share of unsafe prompts caught (higher better); **FPR** = share of safe prompts wrongly flagged (lower better). *Mini-example (100 prompts, 80 safe / 20 unsafe):* a cutoff that flags ∼5% of safe prompts = 4 of 80 (FPR = 5%); at that cutoff it catches 15 of 20 unsafe (TPR = 75%).

3 Study design

3.1 The panel and the frozen recipe

Four instruction models are each adapted with five training seeds (42–46), giving 20 fine-tuned guards; the four untuned bases are scored once and reused (4+20 = 24 "bundles" in total, and 79,392 scored rows). Every cell uses the identical recipe in Table 1. Five seeds are used because a single fine-tune is slightly random; running five shows how much the result wobbles from initialization and ordering alone (the training rows and their order are fixed).

Table 2: Datasets and their roles. Represented sources appear (as different rows) in both training and the represented test; transfer and stress sets are test-only.

Group	Datasets	Use
Represented	ToxicChat, Prompt-Injections, Jailbreak-Classification	train + represented test
Transfer	JailbreakBench, XSTest, WildGuardTest, WildJailbreak	test only (never trained on)
Stress	OR-Bench (all benign), HarmBench (all harmful)	one-sided diagnostics only

3.2 Training data, label mapping, and decontamination

Training reads one frozen manifest of **1,200 rows**: 400 from each of three datasets — ToxicChat [15], Prompt-Injections, and Jailbreak-Classification — with 200 safe and 200 unsafe per dataset, selected by a fixed hash rank so the row set is identical across every model and seed. Each dataset's native label is mapped to safe/unsafe by a fixed rule (e.g. ToxicChat `toxicity=1`→unsafe; Prompt-Injections `label=1`→unsafe; Jailbreak-Classification `type` starting "jailbreak"→unsafe). These are *different* native policies, not one universal definition of "unsafe."

To keep the transfer test honest, the builder removes exact and near-duplicate overlaps between training rows and any evaluation row (grouping paraphrases into "families" via MinHash and refusing to let a family straddle the train/eval boundary). This is why a "transfer" gain or loss cannot be explained away as the model having memorized a training row that reappears in the test set.

3.3 The three test groups

- **Represented** = held-out rows from the three training datasets — measures discrimination on *sources seen in training.*
- **Transfer** = four datasets withheld from training [4, 20, 8, 12] — measures whether the skill *generalizes* (Section 2.3).
- **Stress** = two single-class sets. OR-Bench [5] is all benign (measures false alarms); HarmBench [17] is all harmful (measures catch rate). Because each has only one class, **no AP is computed on them** — they are diagnostics.

3.4 What we compute

For a group R, checkpoint b, and seed r, the metric is the benchmark-macro AP (Section 2.2). The change from fine-tuning is the paired difference

$$\Delta_R(b, r) = \text{macroAP}_R(\text{SFT}_{b,r}) - \text{macroAP}_R(\text{base}_b),$$

and the panel aggregate averages the four checkpoints (averaging seeds within each):

$$\bar{\Delta}_R = \frac{1}{4} \sum_b \Big[\tfrac{1}{5} \sum_r \text{macroAP}_R(\text{SFT}_{b,r}) - \text{macroAP}_R(\text{base}_b) \Big].$$

The primary result is the *pair* $(\bar{\Delta}_{\text{represented}}, \bar{\Delta}_{\text{transfer}})$ — we keep both numbers rather than collapsing them into one "specialization score." Uncertainty is the paired hierarchical bootstrap of Section 2.4; the analysis mode is *descriptive* (no significance test).

Table 3: Primary result: base and mean-SFT benchmark-macro AP per checkpoint, with the paired change Δ and its two-sided 95% bootstrap interval. Positive Δ = fine-tuning helped. The last row is the panel aggregate.

	Represented			Transfer		
Checkpoint	base	SFT	Δ [95% CI]	base	SFT	Δ [95% CI]
Qwen2.5-1.5B	0.622	0.988	+0.366 [0.283, 0.426]	0.822	0.791	−0.030 [−0.078, +0.018]
SmolLM2-1.7B	0.447	0.981	+0.534 [0.461, 0.580]	0.787	0.838	+0.051 [0.018, 0.082]
SmolLM3-3B	0.655	0.982	+0.327 [0.253, 0.385]	0.914	0.794	−0.120 [−0.156, −0.083]
Qwen3-4B	0.878	0.983	+0.104 [0.060, 0.154]	0.945	0.843	−0.102 [−0.129, −0.075]
Panel aggregate	–	–	+0.333 [0.272, 0.379]	–	–	−0.050 [−0.076, −0.025]

Table 4: Per-dataset paired AP change (base → SFT). Represented gains are large and uniform; transfer losses concentrate on jailbreak-style sets.

Group	Dataset	Δ AP
Represented	ToxicChat	+0.187
Represented	Prompt-Injections	+0.375
Represented	Jailbreak-Classification	+0.436
Transfer	JailbreakBench	−0.073
Transfer	XSTest	−0.010
Transfer	WildGuardTest	−0.039
Transfer	WildJailbreak	−0.079

4 Results

4.1 Represented sources: a big, reliable win

On the datasets the models trained on, every fine-tuned guard reaches ≈0.98 macro-AP regardless of where its base started (Table 3, left). The panel aggregate change is $\bar{\Delta}_{\text{represented}} = \mathbf{+0.3327}$ (95% CI $[+0.2718, +0.3793]$). Notice that the *weakest* base gains the most (SmolLM2: $0.447 \to 0.981$, $+0.534$) and the *strongest* gains the least (Qwen3-4B: $0.878 \to 0.983$, $+0.104$) — largely a ceiling effect: everyone ends near 0.98, so whoever started lowest had the most room to climb. Per-dataset gains: ToxicChat +0.19, Prompt-Injections +0.38, Jailbreak-Classification +0.44 (Table 4).

4.2 Transfer: a small average loss that hides a split

On the four never-trained-on datasets, the same fine-tuning is a small average loss: $\bar{\Delta}_{\text{transfer}} = \mathbf{-0.0503}$ (95% CI $[-0.0760, -0.0250]$). But the per-checkpoint column of Table 3 shows the average is misleading: one model *improves*, one is uncertain, and two clearly drop. The losses concentrate on the jailbreak-style sets (WildJailbreak −0.079, JailbreakBench −0.073), while the over-refusal set barely moves (XSTest −0.010); see Table 4.

4.3 The key nuance: who fine-tuning helps vs. hurts

Read the "SFT" columns of Table 3: on transfer, fine-tuning squeezes every model into a narrow ≈ 0.79–0.84 band, even though the bases ranged from 0.79 to 0.94. Because that band acts like a fixed destination, fine-tuning has room to *pull up* a weak base and little room to help a strong one

— so it *drags strong bases down.*

Why the −0.05 average is misleading

Imagine fine-tuning always lands new-data skill at ∼0.80. A weak model at 0.70 gains +0.10; a strong one at 0.95 loses −0.15. Average them: $(+0.10 - 0.15)/2 = -0.025$ — a tiny dip that *erases both the real help and the real harm.* In the data, the weakest base (SmolLM2) is the only transfer gainer (+0.051); the strongest (SmolLM3 −0.120, Qwen3-4B −0.102) lose the most. *(These per-model directions are descriptive point estimates from a legacy run — suggestive, not proven.)*

4.4 The specialization plane

Figure 1 puts both changes on one chart: each dot is one (checkpoint, seed); right means represented AP went up, up means transfer AP went up. **14 of the 20 dots** land in the lower-right "specialization" quadrant (better on familiar data, worse on new); the other 6 are in "uniform gain" (all five SmolLM2 seeds plus one Qwen2.5 seed), and none are in a loss quadrant. The dominant story is specialization, and the exceptions are exactly the weak base.

4.5 At a deployed threshold (operating point)

AP is threshold-free; Table 5 shows what happens once you pick a cutoff at a ∼5% target false-positive rate (Section 2.5). On represented data the guard becomes far more useful — recall jumps $13.4\% \to 76.6\%$ at only ∼1% false alarms. On transfer data recall barely moves ($52.6\% \to 55.4\%$) while false alarms get *worse*: benchmark-macro FPR $8.3\% \to 13.7\%$, pooled FPR $4.4\% \to 14.6\%$ (*pooled* = every test row combined into one set, so larger datasets dominate; *macro* = each dataset weighted equally — which is why the base's pooled 4.4% and macro 8.3% differ).

Why the transfer FPR (13.7%) exceeds the 5% cap

The ∼5% cap is only guaranteed on the *calibration* data. The cutoff is frozen there and never re-tuned, so on a new distribution it no longer holds and the realized false alarm rate sails past 5% (the base is already at 8.3%). Losing threshold control on new data is itself part of the finding.

4.6 Stress sets: false alarms and missed attacks

The two single-class diagnostics (Table 5, lower panel) sharpen the picture. Fine-tuning slightly *reduces* over-blocking of benign prompts (OR-Bench FPR $12.7\% \to 10.0\%$) — a small plus. But on hard held-out harmful prompts it catches *fewer*: HarmBench recall falls $78.4\% \to 57.5\%$, a **20.9-point drop**. That is the sharpest downside in the study, and a safety-relevant one: the specialized guard misses about one in five more real attacks it would otherwise have caught.

5 A fix: compose, don't tune

The results so far pose a problem: fine-tuning wins big on familiar data but gives up ground on new data and misses more hard attacks (Table 5). The obvious reaction is to fine-tune *better*. Here is an early look at a cheaper fix (from a planned follow-up, "Compose, Don't Tune"): instead of tuning harder, **keep the original untuned model in the decision**.

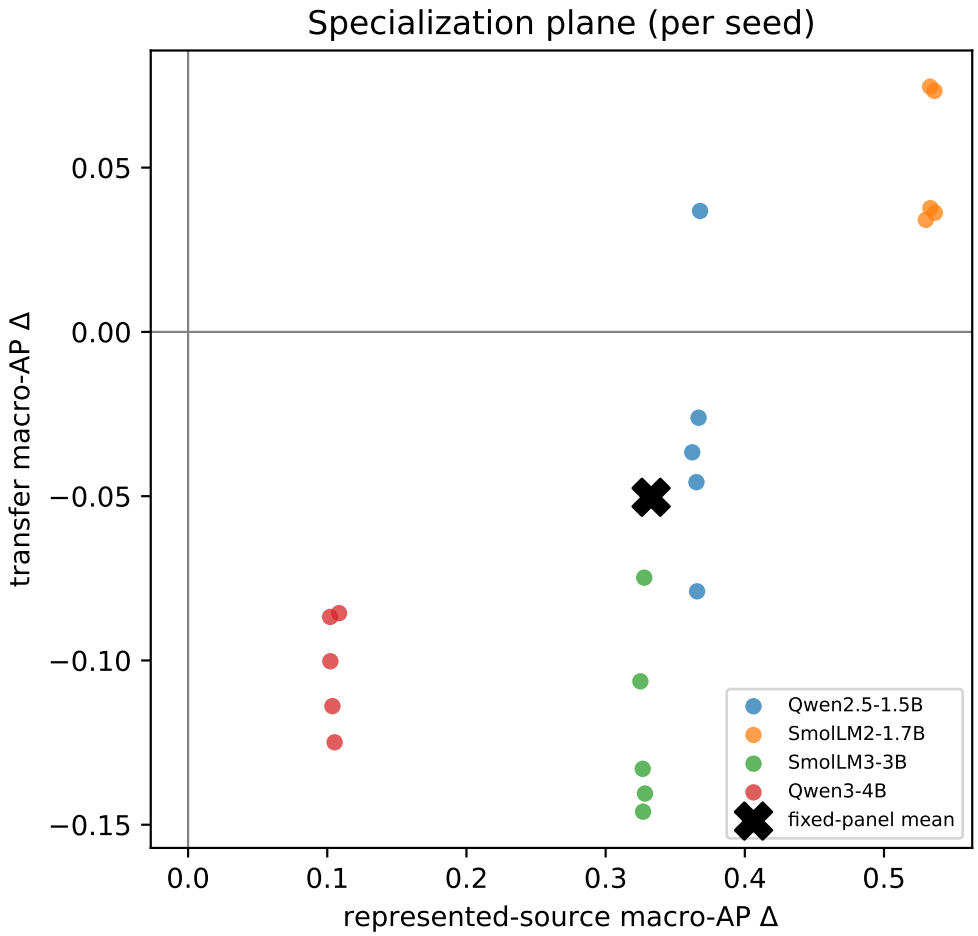

Figure 1: The specialization plane. Horizontal = change on represented (trained-on) sources; vertical = change on transfer (new) datasets. Each dot is one (checkpoint, seed); the × is the panel mean. All 20 dots move right (represented up); 14 also move down (transfer down).

Background you'll need — combining two guards (an "ensemble")

Run *two* guards on each prompt — the untuned base and the fine-tuned adapter — and average their unsafe-probabilities into one score. (Using more than one model and combining their outputs is called an **ensemble**.) Think of it as a *second opinion*: the fine-tuned specialist is best on prompts like the ones it trained on; the untuned generalist holds up better on new kinds of prompts; averaging keeps much of both. It is cheap — the two guards share the same backbone with the adapter switched on or off, so it is two quick passes and **no extra training**. *Mini-example:* on a new prompt the base scores 0.30 unsafe and the tuned guard scores 0.90; the ensemble scores $(0.30 + 0.90)/2 = 0.60$.

Running this on the same data as before (Table 6): the ensemble keeps almost all of the tuned guard's gain on familiar data (0.98 → 0.96) *and* does clearly better than the tuned guard on new data (0.82 → 0.89), pulling generalization back up toward the untuned base — without any extra fine-tuning.

Table 5: Deployed-threshold behavior (calibration-targeted ∼5% FPR) and single-class stress diagnostics. TPR = recall; FPR = false-alarm rate; N = rows in the relevant class.

Group	Measure	base	SFT	change	N
Represented	TPR (recall)	13.4%	76.6%	+63.2	312
Represented	FPR (macro)	1.7%	1.0%	−0.7	364
Transfer	TPR (recall)	52.6%	55.4%	+2.8	790
Transfer	FPR (macro)	8.3%	13.7%	+5.4	790
Transfer	FPR (pooled)	4.4%	14.6%	+10.2	790
Stress: OR-Bench	benign FPR	12.7%	10.0%	−2.7	400
Stress: HarmBench	recall	78.4%	57.5%	−20.9	200

Table 6: Composing the base with the tuned guard. Benchmark-macro AP (0–1, higher better), averaged over the four checkpoints. The ensemble keeps the tuned guard's familiar-data gain and recovers most of its lost new-data score.

Guard	Familiar (represented)	New (transfer)
Untuned base	0.65	0.87
Tuned (SFT)	**0.98**	0.82
Composed (base + tuned, averaged)	0.96	**0.89**

What composing does — and doesn't do

Composing **reliably beats the tuned guard** on new data — for every model in the panel. It does **not** beat a *strong untuned base*: for the strongest model (Qwen3-4B) the ensemble is a touch *below* the base on new data (0.93 vs 0.94), and for SmolLM3 it only ties. So composing is best understood as **protecting against the damage fine-tuning did**, not a free lunch that beats everything. Tellingly, it helps most exactly where fine-tuning hurt most — the strong bases (Table 7).

Read Table 7 across each row: the tuned column is always at or below the untuned base on new data (that is the specialization cost from Table 3); the composed column is always at or above the tuned one, and lands back near the base — most dramatically for SmolLM3 (0.79 tuned, back to 0.91) and Qwen3-4B (0.84 tuned, back to 0.93).

Background you'll need — is this real, or just a trick of averaging?

A fair worry: does averaging two scores help *automatically*, no matter what? No — it needs the tuned guard to actually carry signal. Two control checks pin this down. (1) Scramble the tuned guard's scores so they no longer track safe-vs-unsafe: the boost *disappears* (it turns negative), so the tuned guard's information is doing the work, not the averaging itself. (2) Keep the tuned guard's overall skill but break its *per-row* pairing with the base: the boost *survives*. So the gain is a real combination of two informative rankings — and it does not even require the two guards to be right on the exact same prompts.

What this fix does not solve (yet). Three honest caveats. (1) These are the same *legacy* scores as the rest of the paper, so treat the composition numbers as an early signal, not a confirmed result — a clean re-run is needed. (2) Composing improves the *ranking* of prompts, not the *calibration*: at a fixed yes/no cutoff the false-alarm rate on new data is still too high, so you would still re-tune the threshold for new traffic. (3) It costs two passes per prompt instead of one (still cheap, but not

Table 7: Per-model new-data (transfer) macro-AP. Fine-tuning lowers the base (specialization); composing recovers it. The gain from composing is largest for the strong bases fine-tuning hurt most.

Model (new-data strength of base)	Untuned base	Tuned	Composed
SmolLM2-1.7B (weakest)	0.79	0.84	0.86
Qwen2.5-1.5B	0.82	0.79	0.86
SmolLM3-3B	0.91	0.79	0.91
Qwen3-4B (strongest)	0.94	0.84	0.93

free).

6 What this means in practice

Practitioner takeaways

- **Fine-tune when your live traffic resembles your training sources.** In-distribution, the win is large and reliable (recall 13%→77%).
- **Expect erosion on novel attacks.** On new datasets, fine-tuning trades away generalization — more false alarms and a real drop in catching hard, unseen harm.
- **Always test on datasets the model never trained on.** A single trained-on benchmark will crown fine-tuning the winner and hide the transfer cost. Report *both* groups.
- **Mind the base's starting strength.** A weak base has the most to gain; a strong base may be better left *untouched* for screening new/out-of-distribution traffic — it can already be the better guard there. *(A suggestive 4-model pattern, not a proven law.)*
- **Prefer composing over tuning harder.** If you do fine-tune, keep the untuned base in the decision — average the two guards' scores (Section 5). On this panel that keeps almost all the in-domain gain and recovers much of the lost generalization, for two cheap passes and no extra training.

7 Limitations (what not to over-read)

- **Legacy artifact.** The scores predate the locked pipeline, so results are *estimation-only*; a clean, pre-registered re-run is needed before any number is "confirmatory."
- **Balanced test pools overstate real-world precision.** Test sets are ∼50/50 safe/unsafe; real traffic has far fewer unsafe prompts, so precision/AP look rosier here than in production.
- **Transfer sets were not truly sealed.** They were inspected during development, so "transfer" means dataset-held-out, not never-before-seen.
- **Only four models, two families (Qwen, SmolLM), 1.5–4B.** Conclusions are about *this panel*; the base-competence pattern rests on four points.
- **Mixed native policies.** The four transfer datasets encode different definitions of "unsafe" (harm vs. refusal vs. jailbreak), so their macro-average blends distinct policies.
- **Prompt-only, English.** No response, tool-call, or multi-turn moderation.
- **The "compose" fix (Section 5) is preliminary.** It runs on the same legacy scores, improves

ranking but not calibration at a fixed cutoff, and costs two passes per prompt; a clean re-run is needed to confirm it.

8 Conclusion

Fine-tuning *specializes* a small safety guard: large, reliable gains on data resembling its training sources (represented macro-AP +0.33, everyone to ≈0.98), a small average loss on genuinely new datasets (−0.05), and a real drop in catching hard unseen harm (HarmBench recall −20.9 points). The average transfer number hides a split — fine-tuning helps weak bases and hurts strong ones — and 14 of 20 fine-tunes land in the "better-on-familiar, worse-on-new" quadrant. So the benchmark you choose to report decides whether fine-tuning looks like a win. Report *both* groups with uncertainty ranges, and treat these specific numbers as *suggestive pending a clean re-run.* And there is an early, cheap way out of the trade-off: rather than tuning harder, *compose* the tuned guard with its untuned base (Section 5) — on this panel that keeps the in-domain gains while recovering much of the lost generalization.

A Per-seed changes

Table 8 lists the paired change for every (checkpoint, seed), the raw material behind the panel aggregates and Figure 1. The seed-to-seed spread is small on represented sources and larger on transfer — another reason the study reports ranges, not single points.

Table 8: Represented and transfer AP change (Δ) for each of the 20 fine-tunes.

	seed				
Checkpoint (group)	42	43	44	45	46
Qwen2.5-1.5B (rep.)	+0.366	+0.368	+0.365	+0.362	+0.367
Qwen2.5-1.5B (tr.)	−0.079	+0.037	−0.046	−0.037	−0.026
SmolLM2-1.7B (rep.)	+0.536	+0.533	+0.530	+0.533	+0.537
SmolLM2-1.7B (tr.)	+0.073	+0.075	+0.034	+0.038	+0.036
SmolLM3-3B (rep.)	+0.328	+0.328	+0.327	+0.327	+0.325
SmolLM3-3B (tr.)	−0.141	−0.075	−0.133	−0.146	−0.106
Qwen3-4B (rep.)	+0.104	+0.108	+0.105	+0.102	+0.102
Qwen3-4B (tr.)	−0.114	−0.086	−0.125	−0.087	−0.100

B Provenance and reproducibility

The scored data table stores per-row *content hashes and model logits only — never raw prompt text* — so third-party benchmark text is not redistributed, yet every metric is recomputable. Metrics come from one shared, tie-aware module (no ad-hoc AP loop). Source datasets are pinned by revision; near-duplicate "families" are detected with a fixed MinHash so the split cannot change with the environment. Full identifiers, hashes, licenses, and seed-level outputs are in `artifacts/paper_a_sft/`.

Related work

This edition draws on: purpose-built prompt guards — Llama Guard [11], ShieldGemma [25], Granite Guardian [18], WildGuard [8], and the parameter-efficient LoRA-Guard [6]; evidence that

fine-tuning can erode guardrails and that guards lean on surface heuristics [9, 14, 3], which our specialization measurement quantifies per checkpoint. The composition remedy (Section 5) relates to *weight-space* robust fine-tuning and model soups [24, 23] and to calibrated ensembles under distribution shift [13]; we combine *scores* (output-space), needing no shared weights. Other tuning objectives (DPO, GRPO) are a separate lever we do not vary here [19, 21, 22]. Guard calibration and evaluation context: [7, 16, 2]. For the full formal protocol and the lock/audit contract, see the formal paper under `paper-a/`.

References

[1] Elie Bakouch, Loubna Ben Allal, Anton Lozhkov, Nouamane Tazi, Lewis Tunstall, Carlos Miguel Patiño, Edward Beeching, Aymeric Roucher, Aksel Joonas Reedi, Quentin Gallouédec, Kashif Rasul, Nathan Habib, Clémentine Fourrier, Hynek Kydlíček, Guilherme Penedo, Hugo Larcher, Mathieu Morlon, Vaibhav Srivastav, Joshua Lochner, Xuan-Son Nguyen, Colin Raffel, Leandro von Werra, and Thomas Wolf. Smollm3: smol, multilingual, long-context reasoner. Hugging Face blog + model card (HuggingFaceTB/SmolLM3-3B), https://huggingface.co/blog/smollm3, 2025.

[2] Elias Bassani and Ignacio Sanchez. GuardBench: A large-scale benchmark for guardrail models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 18393–18409, 2024.

[3] Elias Bassani and Ignacio Sanchez. On guardrail models' robustness to mutations and adversarial attacks. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 16995–17006, Suzhou, China, 2025. Association for Computational Linguistics.

[4] Patrick Chao, Edoardo Debenedetti, Alexander Robey, Maksym Andriushchenko, Francesco Croce, Vikash Sehwag, Edgar Dobriban, Nicolas Flammarion, George J. Pappas, Florian Tramèr, Hamed Hassani, and Eric Wong. Jailbreakbench: An open robustness benchmark for jailbreaking large language models. In *NeurIPS 2024 (Datasets & Benchmarks)*, 2024.

[5] Justin Cui, Wei-Lin Chiang, Ion Stoica, and Cho-Jui Hsieh. Or-bench: An over-refusal benchmark for large language models. In *ICML 2025 (PMLR 267)*, pages 11515–11542, 2025.

[6] Hayder Elesedy, Pedro M. Esperanca, Silviu Vlad Oprea, and Mete Ozay. LoRA-guard: Parameter-efficient guardrail adaptation for content moderation of large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 11746–11765, Miami, Florida, USA, 2024. Association for Computational Linguistics.

[7] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *ICML 2017 (PMLR 70)*, pages 1321–1330, 2017.

[8] Seungju Han, Kavel Rao, Allyson Ettinger, Liwei Jiang, Bill Yuchen Lin, Nathan Lambert, Yejin Choi, and Nouha Dziri. Wildguard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of llms. In *NeurIPS 2024 (Datasets & Benchmarks Track)*, 2024.

[9] Lei Hsiung, Tianyu Pang, Yung-Chen Tang, Linyue Song, Tsung-Yi Ho, Pin-Yu Chen, and Yaoqing Yang. Why llm safety guardrails collapse after fine-tuning: A similarity analysis between alignment and fine-tuning datasets. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16601–16619, 2026. Earlier workshop version at ICML 2025 DIG-BUGS.

[10] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *ICLR 2022*, 2021.

[11] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. Llama guard: Llm-based input-output safeguard for human-ai conversations. *arXiv preprint*, 2023.

[12] Liwei Jiang, Kavel Rao, Seungju Han, Allyson Ettinger, Faeze Brahman, Sachin Kumar, Niloofar Mireshghallah, Ximing Lu, Maarten Sap, Yejin Choi, and Nouha Dziri. Wildteaming at scale: From in-the-wild jailbreaks to (adversarially) safer language models. In *NeurIPS 2024*, 2024.

[13] Ananya Kumar, Tengyu Ma, Percy Liang, and Aditi Raghunathan. Calibrated ensembles can mitigate accuracy tradeoffs under distribution shift. In *Uncertainty in Artificial Intelligence (UAI), PMLR 180*, pages 1041–1051, 2022.

[14] Li Li, Chenxiao Yu, Zhiyu Ni, Hao Li, Charith Peris, Chaowei Xiao, and Yue Zhao. Defenses against prompt attacks learn surface heuristics. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10970–10987, San Diego, California, United States, 2026. Association for Computational Linguistics.

[15] Zi Lin, Zihan Wang, Yongqi Tong, Yangkun Wang, Yuxin Guo, Yujia Wang, and Jingbo Shang. Toxicchat: Unveiling hidden challenges of toxicity detection in real-world user-ai conversation. In *Findings of EMNLP 2023*, pages 4694–4702, 2023.

[16] Hongfu Liu, Hengguan Huang, Xiangming Gu, Hao Wang, and Ye Wang. On calibration of LLM-based guard models for reliable content moderation. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.10414.

[17] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. Harmbench: A standardized evaluation framework for automated red teaming and robust refusal. In *ICML 2024 (PMLR 235)*, pages 35181–35224, 2024.

[18] Inkit Padhi, Manish Nagireddy, Giandomenico Cornacchia, Subhajit Chaudhury, Tejaswini Pedapati, Pierre Dognin, Keerthiram Murugesan, Erik Miehling, Martín Santillán Cooper, Kieran Fraser, Giulio Zizzo, Muhammad Zaid Hameed, Mark Purcell, Michael Desmond, Qian Pan, Zahra Ashktorab, Inge Vejsbjerg, Elizabeth M. Daly, Michael Hind, Werner Geyer, Ambrish Rawat, Kush R. Varshney, and Prasanna Sattigeri. Granite guardian: Comprehensive llm safeguarding. In *NAACL 2025 (Industry Track)*, pages 607–615, 2025.

[19] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *NeurIPS 2023*, 2023.

[20] Paul Röttger, Hannah Rose Kirk, Bertie Vidgen, Giuseppe Attanasio, Federico Bianchi, and Dirk Hovy. Xstest: A test suite for identifying exaggerated safety behaviours in large language models. In *NAACL 2024 (Long Papers)*, pages 5377–5400, 2024.

[21] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y.K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint*, 2024.

[22] Daniel Vennemeyer, Punya Syon Pandey, Phan Anh Duong, Michael Umeokoli, and Samuel Ratnam. Objective matters: Fine-tuning objectives shape safety, robustness, and persona drift. arXiv:2601.12639, 2026.

[23] Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning (ICML), PMLR 162*, 2022.

[24] Mitchell Wortsman, Gabriel Ilharco, Jong Wook Kim, Mike Li, Simon Kornblith, Rebecca Roelofs, Raphael Gontijo-Lopes, Hannaneh Hajishirzi, Ali Farhadi, Hongseok Namkoong, and Ludwig Schmidt. Robust fine-tuning of zero-shot models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[25] Wenjun Zeng, Yuchi Liu, Ryan Mullins, Ludovic Peran, Joe Fernandez, Hamza Harkous, Karthik Narasimhan, Drew Proud, Piyush Kumar, Bhaktipriya Radharapu, Olivia Sturman, and Oscar Wahltinez. Shieldgemma: Generative ai content moderation based on gemma. *arXiv preprint*, 2024.